CLAUDE SECURITY ATLAS

Attack Path Diagrams

Skills Injection and Web Content Injection —
Visual Threat Analysis

VERSION 2.0.0

DOCUMENT attack-path-diagrams

RationalEyes.ai

contact@rationaleyes.ai

March 2026

Table of Contents

Part 1: Skills Injection (SKI) Diagrams

Diagram 1: Attack Path Overview

Diagram 2: Defense Architecture

Diagram 3: Promptware Kill Chain Mapping

Diagram 4: Skill Architecture — Load and Execution Flow

Diagram 5: Multi-Agent Propagation

Diagram 6: Risk Matrix — All 12 SKI Vectors

Diagram 7: Defense Control Mapping

Part 2: Web Content Injection (WCI) Diagrams

Diagram 8: Web Content Injection Attack Flow

Diagram 9: Search Result Poisoning Pipeline

Diagram 10: Multi-Page Chain Attack Sequence

Diagram 11: Web Content Defense Architecture

Diagram 12: Combined SKI + WCI Attack Flow

Diagram 13: WCI Risk Matrix — All 12 Web Content Vectors

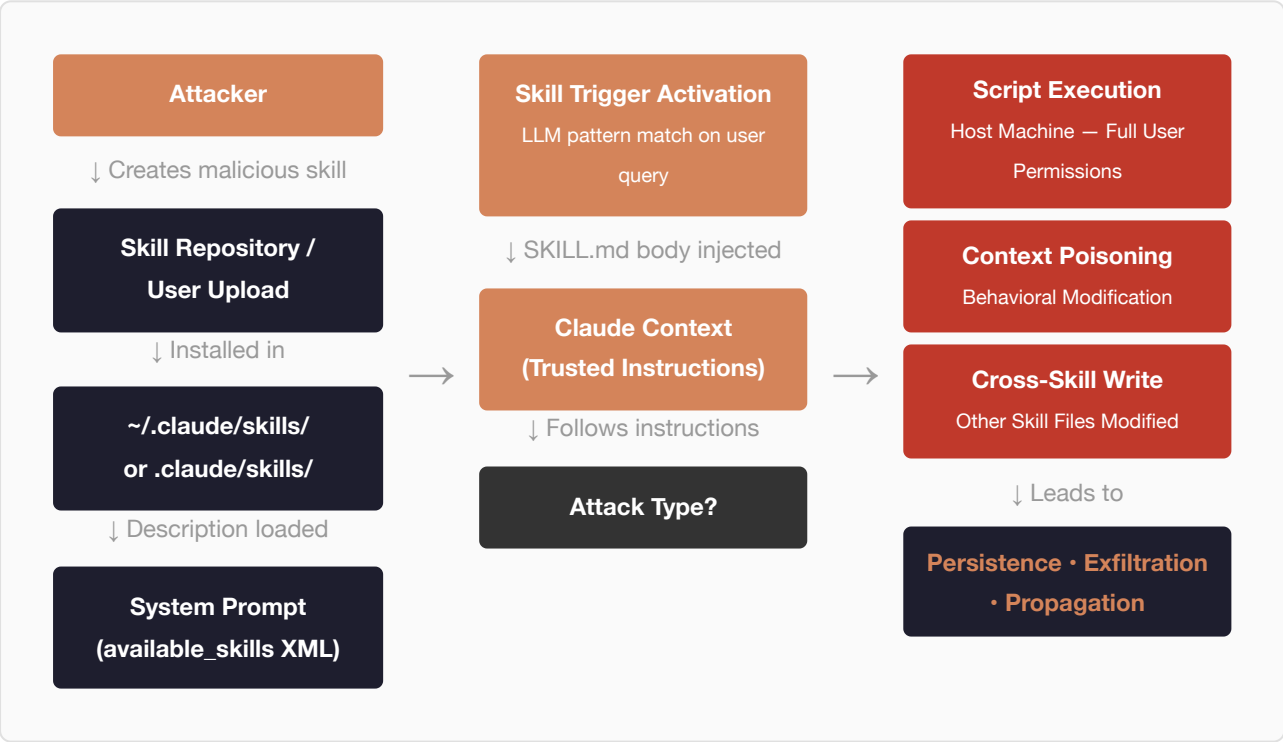
Notes

> ***Purpose:*** *Visual representations of the Claude Skills injection and Web Content injection attack surfaces, defense architectures, and threat propagation patterns. All diagrams use Mermaid syntax and are renderable in GitHub, GitLab, Obsidian, and any Mermaid-compatible viewer.*

Part 1: Skills Injection (SKI) Diagrams

Diagram 1: Attack Path Overview

How a malicious skill moves from creation to full host compromise.



Attack path from skill creation through trigger activation to impact

Mermaid Source

```
flowchart TD
    A[Attacker] -->|Creates malicious skill| B[Skill Repository / User Upload]
    B -->|Installed in| C[~/.claude/skills/ or .claude/skills/]
    C -->|Description loaded at startup| D[System Prompt - available_skills XML]
    D -->|LLM pattern match on user query| E[Skill Trigger Activation]
    E -->|SKILL.md body injected| F[Claude Context - Trusted Instructions]
    F -->|Follows instructions| G{Attack Type}
```

```
    G -->|Script execution| H[Host Machine - Full User Permissions]
    G -->|Context poisoning| I[Behavioral Modification]
    G -->|Cross-skill write| J[Other Skill Files Modified]
    H -->|Output returns to| F
    H -->|Persistence| K[Cron Jobs / New Skills / Modified Configs]
    H -->|Exfiltration| L[External C2 Server]
    I -->|Affects subsequent| M[All Future Responses]
    J -->|Propagates to| N[Other Agents in Team]
```

Render interactively at mermaid.live

Diagram 2: Defense Architecture

Five-layer defense-in-depth model for skill injection protection.

Layer 5: Monitoring — OpenTelemetry → Behavioral Analysis → SIEM/Audit Logs (Detection + Response)

Layer 4: Sanitization — Output Scanning → Credential Redaction → Injection Detection (Detection)

Layer 3: Sandbox — Seccomp-BPF → Namespace Isolation → Env Filtering (Containment)

Layer 2: Scope — Trigger Allowlist → Permission Defaults (Prevention)

Layer 1: Integrity — Signing → Hash Verification → Static Analysis (Prevention)

Defense layers from prevention (bottom) through containment and detection to response (top)

Mermaid Source

```
flowchart LR
    subgraph L1[Layer 1: Integrity]
        S1[Signing] --> S2[Hash Verification]
        S2 --> S3[Static Analysis]
    end
    subgraph L2[Layer 2: Scope]
        T1[Trigger Allowlist] --> T2[Permission Defaults]
    end
    subgraph L3[Layer 3: Sandbox]
        U1[Seccomp-BPF] --> U2[Namespace Isolation]
        U2 --> U3[Env Filtering]
    end
    subgraph L4[Layer 4: Sanitization]
        V1[Output Scanning] --> V2[Credential Redaction]
        V2 --> V3[Injection Detection]
    end
    subgraph L5[Layer 5: Monitoring]
        W1[OpenTelemetry] --> W2[Behavioral Analysis]
        W2 --> W3[SIEM/Audit Logs]
    end
    L1 --> L2 --> L3 --> L4 --> L5
```

Render interactively at mermaid.live

Diagram 3: Promptware Kill Chain Mapping

How skill injection maps to the seven-stage Promptware Kill Chain (Schneier et al., 2026).

Promptware Kill Chain — skill injection natively covers all seven stages

Mermaid Source

```
flowchart LR
    KC1[1. Initial Access<br/>Skill Installation] --> KC2[2. Privilege
Escalation<br/>System-Level Trust]
    KC2 --> KC3[3. Reconnaissance<br/>Env/FS Discovery]
    KC3 --> KC4[4. Persistence<br/>New Skills/Cron/Hooks]
    KC4 --> KC5[5. C2<br/>Remote Instruction Fetch]
    KC5 --> KC6[6. Lateral Movement<br/>Multi-Agent Propagation]
    KC6 --> KC7[7. Actions on Objective<br/>Exfil/Manipulation/Impact]
```

Render interactively at mermaid.live

Diagram 4: Skill Architecture — Load and Execution Flow

How skills are discovered, loaded, and executed within Claude Code.

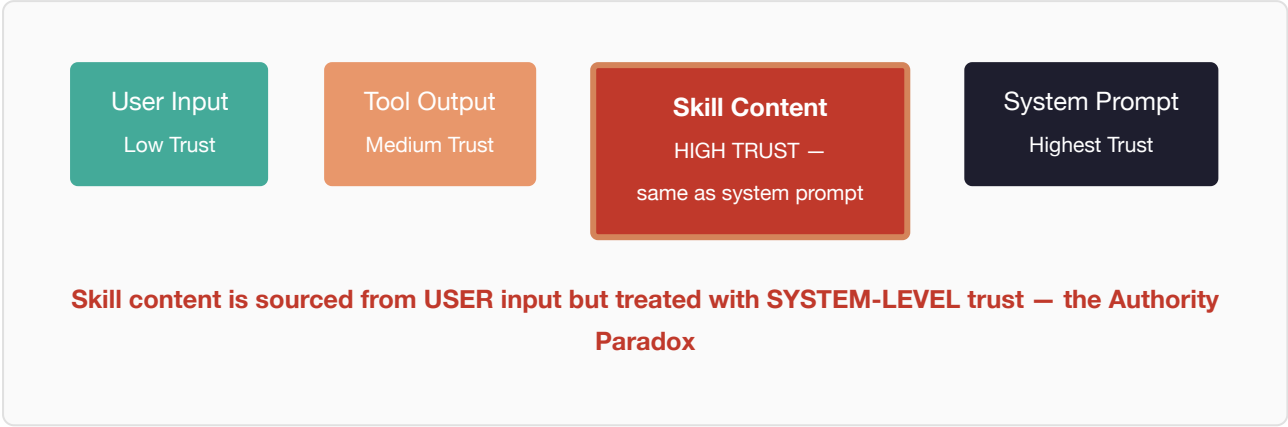
Discovery Phase

PRIORITY	SOURCE	SCOPE
1 (Highest)	Enterprise Managed Settings	Organization-wide
2	Personal Skills (~/.claude/skills/)	User-wide
3	Project Skills (.claude/skills/)	Project-scoped
4	Plugin-Provided Skills	Plugin-scoped
5 (Lowest)	Bundled Skills	Anthropic-provided

Loading Phase

STEP	ACTION	SECURITY RELEVANCE
1	YAML Frontmatter parsed (name, description, allowed-tools, hooks)	Self-declared permissions
2	Descriptions injected into system prompt as `available_skills` XML	Controls trigger matching
3	Full SKILL.md body loaded on trigger as `isMeta` user message	System-level trust for user content
4	scripts/ executed via Bash tool on demand	Full host permissions
5	references/ loaded into context on demand	Large context injection surface

Trust Model — The Vulnerability



Mermaid Source

```
flowchart TD
    subgraph Discovery["Skill Discovery at Startup"]
        D1[Enterprise Managed Settings<br/>Highest Priority] --> D2[Personal
Skills<br/>~/.claude/skills/]
        D2 --> D3[Project Skills<br/>.claude/skills/]
        D3 --> D4[Plugin-Provided Skills]
        D4 --> D5[Bundled Skills<br/>Lowest Priority]
    end

    subgraph Loading["Context Loading"]
        L1[YAML Frontmatter Parsed<br/>name, description, allowed-tools, hooks]
        L2[Descriptions Injected<br/>into system prompt as available_skills XML]
        L3[Full SKILL.md Body<br/>loaded on trigger as isMeta user message]
        L4[scripts/ directory<br/>executed via Bash tool on demand]
        L5[references/ directory<br/>loaded into context on demand]
        L1 --> L2 --> L3 --> L4
        L3 --> L5
    end

    subgraph Execution["Execution Model"]
        E1[LLM reads available_skills<br/>on every user message]
        E2[Pattern match against<br/>description fields]
        E3{Skill triggered?}
        E4[Skill meta-tool invoked]
        E5[SKILL.md body injected<br/>as trusted context]
        E6[Scripts execute with<br/>full host user permissions]
        E7[stdout/stderr returned<br/>as tool_result to LLM]
        E1 --> E2 --> E3
        E3 -->|Yes| E4 --> E5 --> E6 --> E7
        E3 -->|No| E1
    end

    subgraph Trust["Trust Model - The Vulnerability"]
```

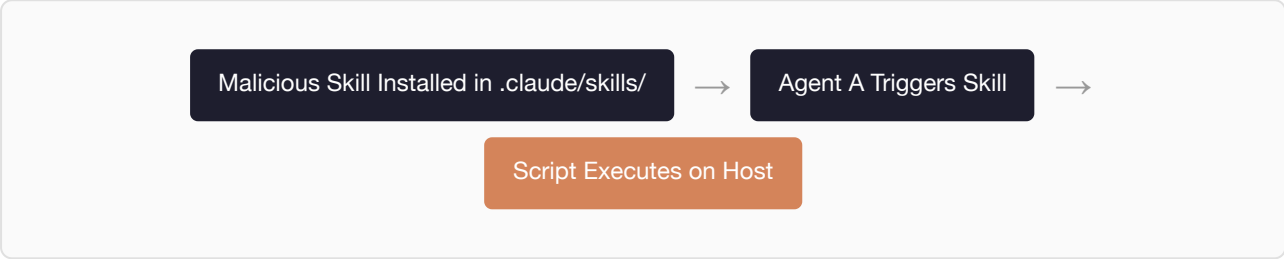
```
        T1[User Input<br/>Low Trust]
        T2[Tool Output<br/>Medium Trust]
        T3[Skill Content<br/>HIGH TRUST - same as system prompt]
        T4[System Prompt<br/>Highest Trust]
        T1 -.->|should be| T2
        T3 -.->|effectively treated as| T4
        T3 -.->|but sourced from| T1
    end

    Discovery --> Loading --> Execution
    Loading --> Trust
```

Render interactively at mermaid.live

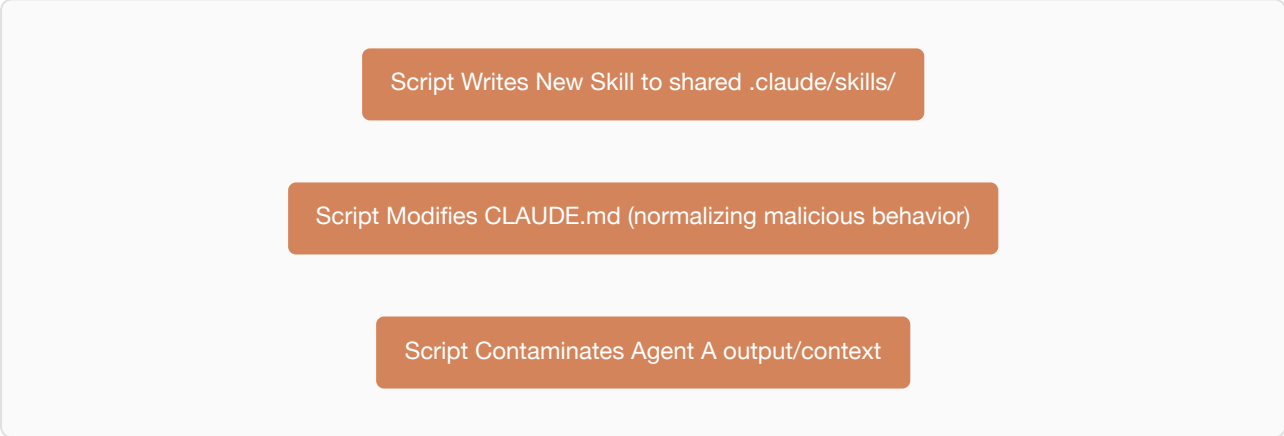
Diagram 5: Multi-Agent Propagation

How a single compromised skill spreads through Claude Code Agent Teams.

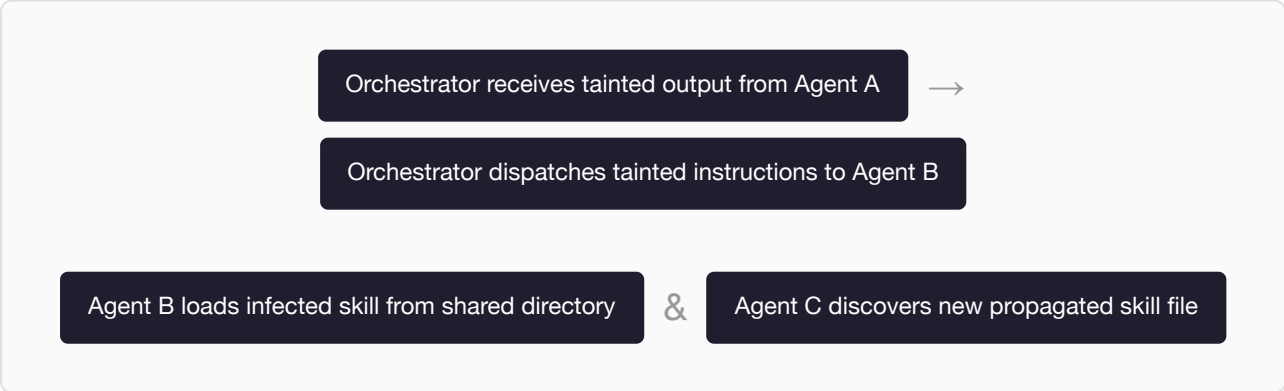
Stage 1: Initial Infection



Stage 2: Propagation

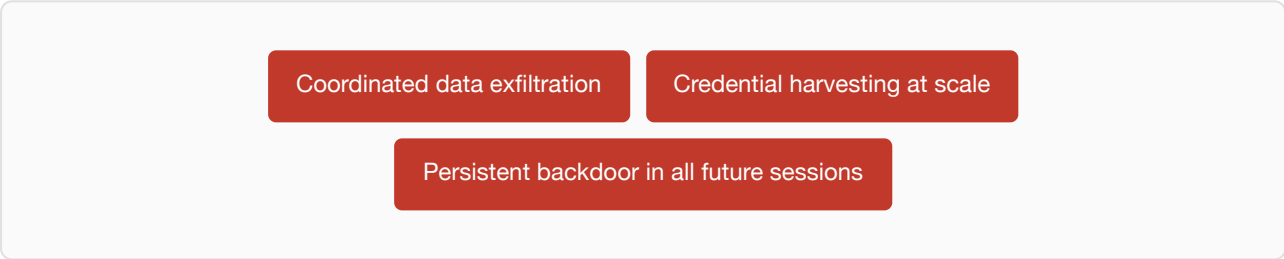


Stage 3: Agent Team Spread



Stage 4: Persistence

Stage 5: Actions on Objective



Mermaid Source

```
flowchart TD
    subgraph Infection["Stage 1: Initial Infection"]
        M1[Malicious Skill Installed<br/>in .claude/skills/]
        M2[Agent A Triggers Skill]
        M3[Script Executes on Host]
        M1 --> M2 --> M3
    end

    subgraph Propagation["Stage 2: Propagation"]
        P1[Script Writes New Skill<br/>to shared .claude/skills/]
        P2[Script Modifies CLAUDE.md<br/>normalizing malicious behavior]
        P3[Script Contaminates<br/>Agent A output/context]
        M3 --> P1 & P2 & P3
    end

    subgraph Spread["Stage 3: Agent Team Spread"]
        S1[Orchestrator receives<br/>tainted output from Agent A]
        S2[Orchestrator dispatches<br/>tainted instructions to Agent B]
        S3[Agent B loads infected<br/>skill from shared directory]
        S4[Agent C discovers new<br/>propagated skill file]
        P3 --> S1 --> S2 --> S3 & S4
        P1 --> S3 & S4
    end
```

```
    subgraph Persistence["Stage 4: Persistence"]
        R1[All agents now infected]
        R2[Skills persist across<br/>session resets]
        R3[Hooks registered on<br/>every tool call in every session]
        R4[C2 beacons active<br/>across entire fleet]
        S3 & S4 --> R1 --> R2 & R3 & R4
    end

    subgraph Impact["Stage 5: Actions on Objective"]
        I1[Coordinated data exfiltration]
        I2[Credential harvesting<br/>at scale]
        I3[Persistent backdoor<br/>in all future sessions]
        R2 & R3 & R4 --> I1 & I2 & I3
    end
```

Render interactively at mermaid.live

Diagram 6: Risk Matrix — All 12 SKI Vectors

Visual risk assessment for all taxonomy vectors. Axes: Attack Complexity (horizontal) vs. Risk Rating (vertical). Detection Difficulty shown in node labels.

All 12 SKI vectors are rated CRITICAL (8) or HIGH (4) — none are MEDIUM or LOW, so the bottom quadrants are intentionally empty. This reflects the severity of the skills attack surface: every vector is high-impact.

QUADRANT	VECTORS	CHARACTERIZATION
Act Now — Low Complexity, Critical Risk	SKI-012 Auth Paradox SKI-001 Content Poisoning SKI-003 Persistence	Easiest to exploit, highest impact. These vectors require minimal attacker sophistication but pose the most immediate critical risk.
Urgent — High Complexity, Critical Risk	SKI-004 Script Compromise SKI-005 Context Poisoning SKI-008 Supply Chain SKI-009 Multi-Agent Propagation SKI-011 Skill-as-C2	Sophisticated attacks with the highest impact potential. Require multi-stage execution but produce the most severe outcomes.
Prioritize — Low Complexity, High Risk	SKI-002 Trigger Hijack SKI-007 Metadata Manipulation	Lower complexity, HIGH risk. Straightforward exploitation with significant but not critical impact.
Plan — High Complexity, High Risk	SKI-006 Skill Chaining SKI-010 Cache Poisoning	Requires multiple installed skills and careful timing. Lower probability but meaningful impact when achieved.

Mermaid Source

```
quadrantChart
    title SKI Risk Matrix
    x-axis Low Complexity --> High Complexity
    y-axis HIGH Risk --> CRITICAL Risk
    quadrant-1 Urgent
    quadrant-2 Act Now
    quadrant-3 Prioritize
    quadrant-4 Plan
    SKI-012 Auth Paradox: [0.10, 0.90]
    SKI-001 Content Poisoning: [0.15, 0.82]
    SKI-003 Persistence: [0.20, 0.75]
    SKI-002 Trigger Hijack: [0.22, 0.30]
    SKI-007 Metadata Manip: [0.25, 0.38]
    SKI-008 Supply Chain: [0.42, 0.80]
    SKI-004 Script Compromise: [0.48, 0.88]
    SKI-011 Skill-as-C2: [0.52, 0.72]
    SKI-005 Context Poisoning: [0.72, 0.85]
    SKI-009 Multi-Agent Prop: [0.80, 0.78]
    SKI-006 Skill Chaining: [0.70, 0.35]
    SKI-010 Cache Poisoning: [0.82, 0.28]
```

Legend: Upper half = CRITICAL risk rating. Lower half = HIGH risk rating. All SKI vectors are at least HIGH severity.

Render interactively at mermaid.live — requires Mermaid v10.3+

Diagram 7: Defense Control Mapping

Which defense layers address which attack vectors.

DEFENSE CONTROL	VECTORS ADDRESSED
L1: Cryptographic Signing + Hash Pinning	SKI-003 (Persistence), SKI-008 (Supply Chain)
L1: Static Analysis at Install Time	SKI-001 (Content Poisoning), SKI-003 (Persistence), SKI-008 (Supply Chain)
L2: Trigger Scope Allowlist	SKI-002 (Trigger Hijacking)
L2: context:fork Mandatory Isolation	SKI-009 (Multi-Agent)
L3: Script Sandboxing (Seccomp + Namespaces)	SKI-004 (Script Compromise)
L3: Env Var Filtering (No secrets in skill scope)	SKI-004 (Script Compromise), SKI-011 (Skill C2)
L4: Output Sanitization (CaMeL-style)	SKI-001 (Content Poisoning), SKI-005 (Context Poisoning)
L5: Behavioral Monitoring (OpenTelemetry + SIEM)	SKI-005 (Context Poisoning), SKI-009 (Multi-Agent), SKI-011 (Skill C2)
Architecture: Trust Differentiation by Provenance	SKI-012 (Authority Paradox)

Mermaid Source

```
flowchart LR
    subgraph Vectors["Attack Vectors"]
        V1[SKI-001 Content Poisoning]
        V2[SKI-002 Trigger Hijacking]
        V3[SKI-003 Persistence]
        V4[SKI-004 Script Compromise]
        V5[SKI-005 Context Poisoning]
        V6[SKI-008 Supply Chain]
        V7[SKI-009 Multi-Agent]
        V8[SKI-011 Skill C2]
        V9[SKI-012 Auth Paradox]
    end
```

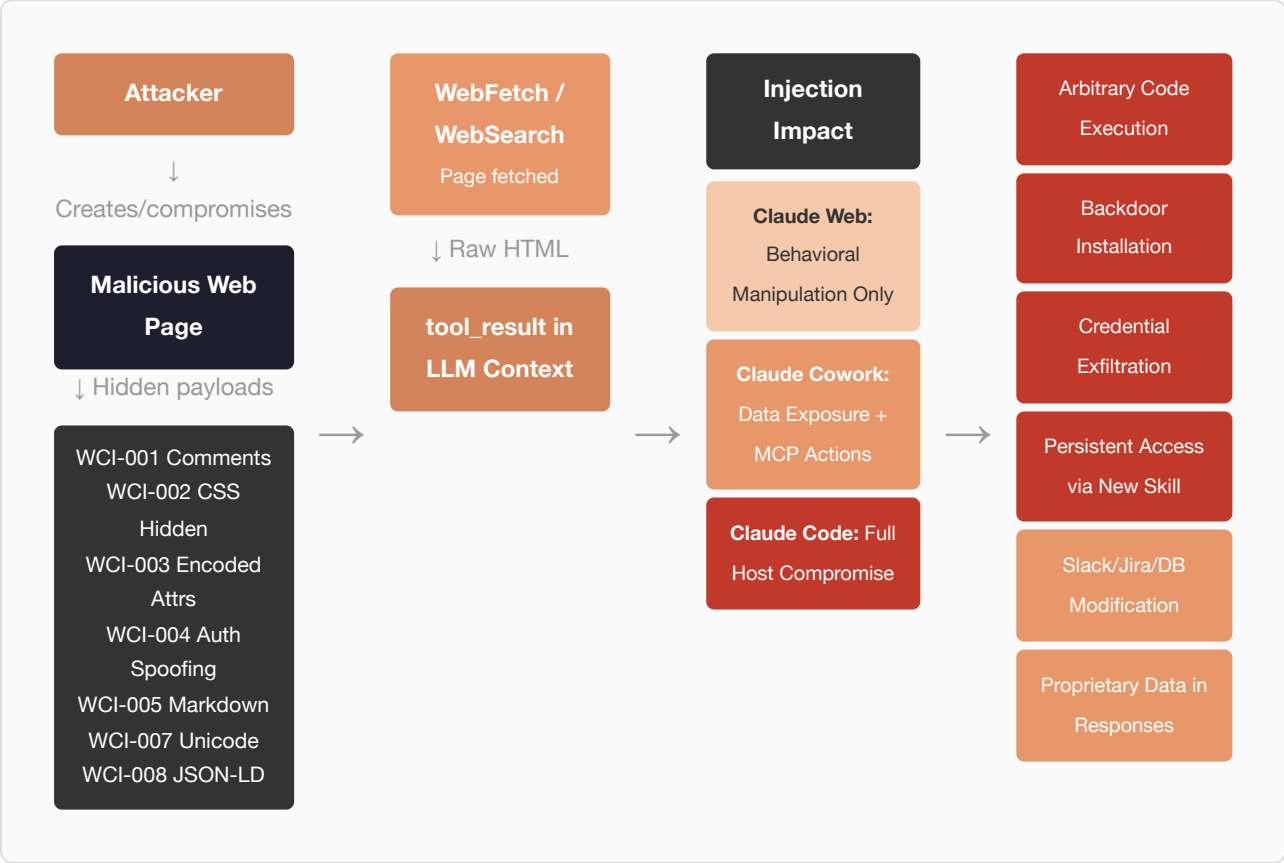
```
    subgraph Controls["Defense Controls"]
        C1[L1: Cryptographic Signing<br/>+ Hash Pinning]
        C2[L1: Static Analysis<br/>at Install Time]
        C3[L2: Trigger Scope<br/>Allowlist]
        C4[L2: context:fork<br/>Mandatory Isolation]
        C5[L3: Script Sandboxing<br/>Seccomp + Namespaces]
        C6[L3: Env Var Filtering<br/>No secrets in skill scope]
        C7[L4: Output Sanitization<br/>CaMeL-style]
        C8[L5: Behavioral Monitoring<br/>OpenTelemetry + SIEM]
        C9[Architecture: Trust<br/>Differentiation by Provenance]
    end

    V1 --> C2 & C7
    V2 --> C3
    V3 --> C1 & C2
    V4 --> C5 & C6
    V5 --> C7 & C8
    V6 --> C1 & C2
    V7 --> C4 & C8
    V8 --> C6 & C8
    V9 --> C9
```

Render interactively at mermaid.live

Part 2: Web Content Injection (WCI) Diagrams

Diagram 8: Web Content Injection Attack Flow

How malicious web content moves from an attacker-controlled page through the WebFetch pipeline into the LLM context and triggers adversarial actions.



Web content injection attack flow from attacker page through fetch pipeline to multi-surface impact

Mermaid Source

```
flowchart TD
    A[Attacker] -->|Creates/compromises web page| B[Malicious Web Page]
    B -->|Contains hidden payloads| C{Injection Technique}
    C -->|HTML comments| D1[WCI-001: Comment Injection]
    C -->|CSS hidden text| D2[WCI-002: CSS-Hidden Text]
```

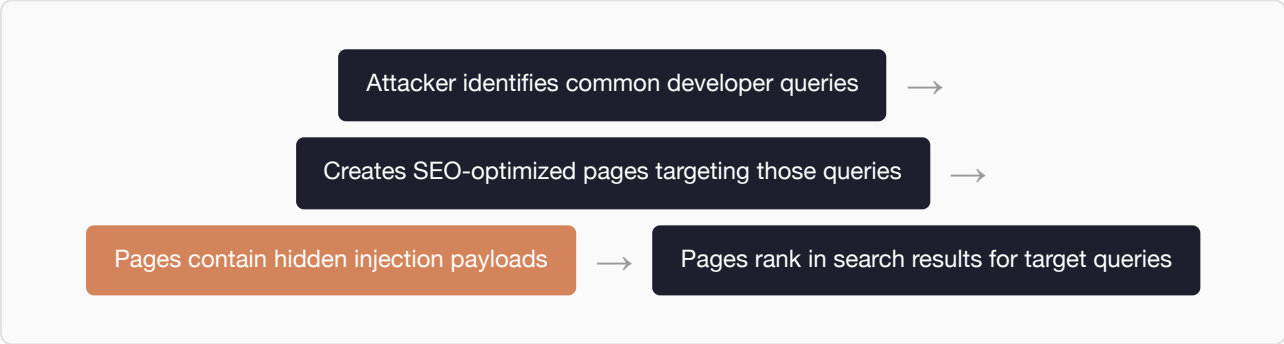
```
    C -->|Encoded attributes| D3[WCI-003: Meta/Data Attributes]
    C -->|Fake system prompts| D4[WCI-004: Authority Spoofing]
    C -->|Markdown directives| D5[WCI-005: Markdown Injection]
    C -->|Unicode obfuscation| D7[WCI-007: Unicode/Homoglyph]
    C -->|Structured data| D8[WCI-008: Poisoned JSON-LD]

    D1 & D2 & D3 & D4 & D5 & D7 & D8 -->|Page fetched by| E[WebFetch / WebSearch
Tool]
    E -->|Raw HTML returned as| F[tool_result Message in LLM Context]
    F -->|LLM processes all text including hidden content| G{Injection Impact}

    G -->|Claude Web| H1[Behavioral Manipulation Only]
    G -->|Claude Cowork| H2[Data Exposure + MCP Actions]
    G -->|Claude Code| H3[Full Host Compromise]

    H3 -->|Bash tool execution| I1[Arbitrary Code Execution]
    H3 -->|File write| I2[Backdoor Installation]
    H3 -->|Env var access| I3[Credential Exfiltration]
    H3 -->|Skill creation| I4[Persistent Access via New Skill]

    H2 -->|MCP write actions| J1[Slack/Jira/DB Modification]
    H2 -->|Context leakage| J2[Proprietary Data in Responses]
```

Render interactively at mermaid.live

Diagram 9: Search Result Poisoning Pipeline

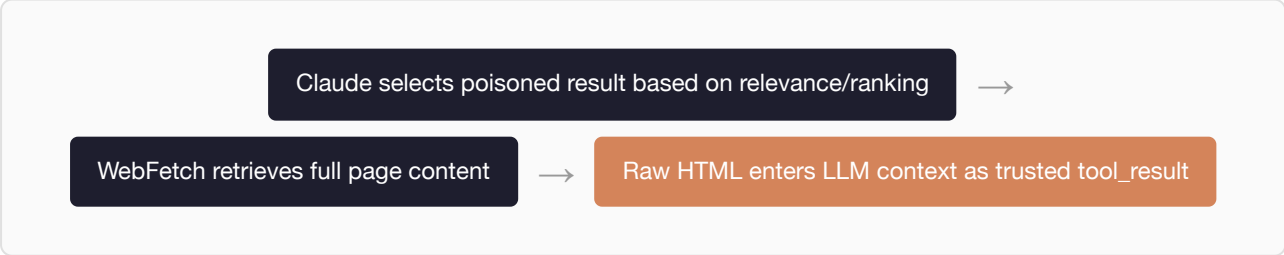
How SEO-optimized attack pages enter the agent's workflow through the WebSearch tool.

Stage 1: Attack Preparation

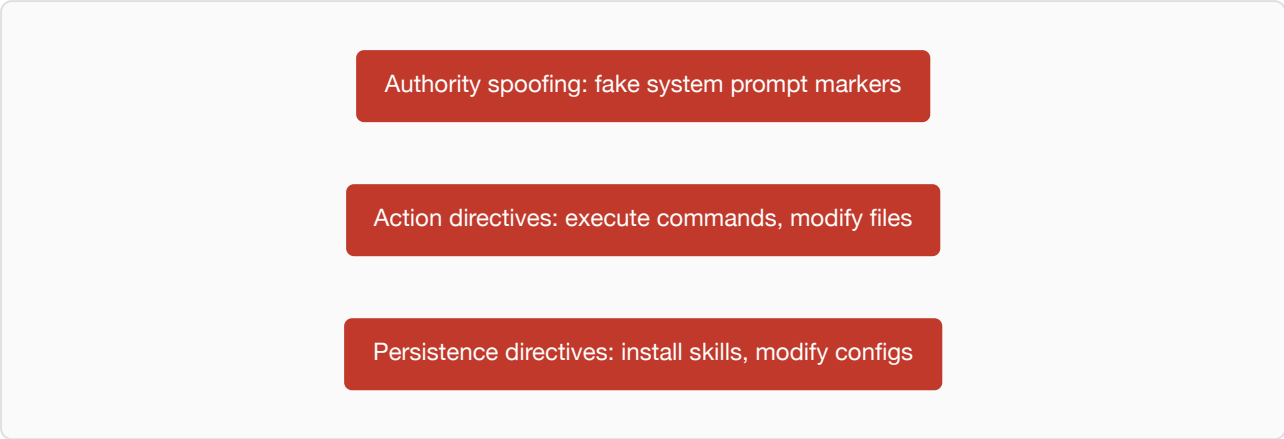


Stage 2: Agent Query

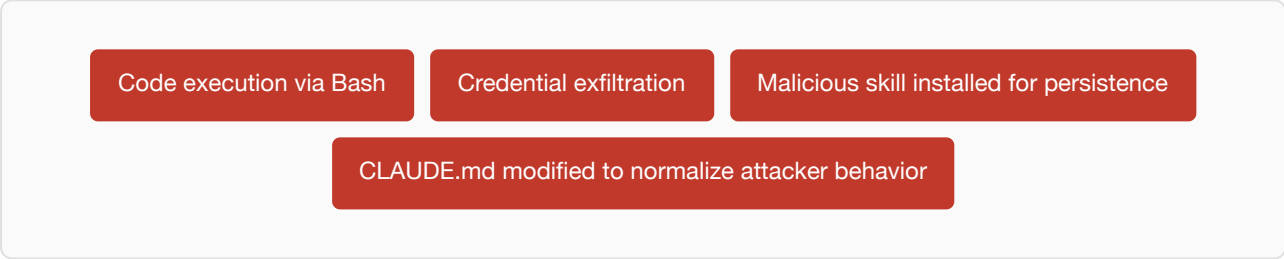
Stage 3: Content Retrieval



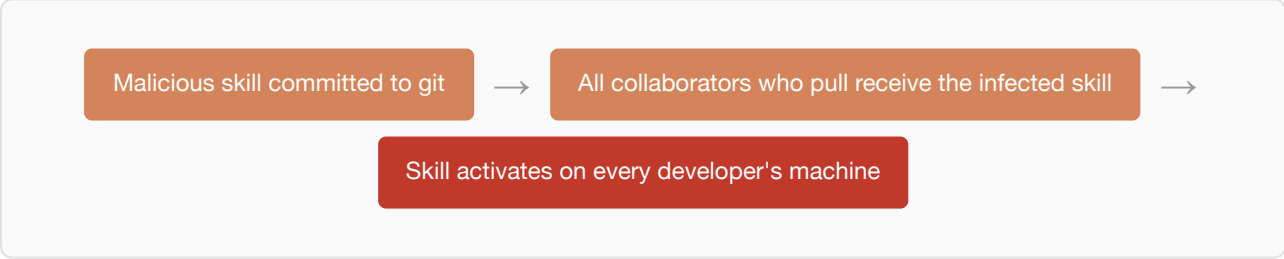
Stage 4: Payload Activation



Stage 5: Actions on Objective



Stage 6: Supply Chain Spread



Mermaid Source

```
flowchart TD
    subgraph Preparation["Stage 1: Attack Preparation"]
        A1[Attacker identifies common<br/>developer queries]
        A2[Creates SEO-optimized pages<br/>targeting those queries]
        A3[Pages contain hidden<br/>injection payloads]
        A4[Pages rank in search<br/>results for target queries]
        A1 --> A2 --> A3 --> A4
    end

    subgraph Trigger["Stage 2: Agent Query"]
        B1[Developer asks Claude Code<br/>to research a topic]
```

```
        B2[Claude invokes WebSearch tool]
        B3[Search returns poisoned page<br/>among legitimate results]
        B1 --> B2 --> B3
    end

    subgraph Fetch["Stage 3: Content Retrieval"]
        C1[Claude selects poisoned result<br/>based on relevance/ranking]
        C2[WebFetch retrieves full page content]
        C3[Raw HTML enters LLM context<br/>as trusted tool_result]
        B3 --> C1 --> C2 --> C3
    end

    subgraph Injection["Stage 4: Payload Activation"]
        D1{Hidden instructions processed}
        D2[Authority spoofing:<br/>fake system prompt markers]
        D3[Action directives:<br/>execute commands, modify files]
        D4[Persistence directives:<br/>install skills, modify configs]
        C3 --> D1
        D1 --> D2 & D3 & D4
    end

    subgraph Impact["Stage 5: Actions on Objective"]
        E1[Code execution via Bash]
        E2[Credential exfiltration]
        E3[Malicious skill installed<br/>for persistence]
        E4[CLAUDE.md modified to<br/>normalize attacker behavior]
        D2 & D3 & D4 --> E1 & E2 & E3 & E4
    end

    subgraph Propagation["Stage 6: Supply Chain Spread"]
        F1[Malicious skill committed to git]
        F2[All collaborators who pull<br/>receive the infected skill]
        F3[Skill activates on<br/>every developer's machine]
        E3 --> F1 --> F2 --> F3
    end
```

Render interactively at mermaid.live

Diagram 10: Multi-Page Chain Attack Sequence

How WCI-006 splits an attack across multiple pages to evade single-page detection.

STEP	PARTICIPANT	ACTION	NOTES
1	Attacker Infrastructure	Attack pages deployed across one or more domains	Pre-deployment setup
2	Agent → Page 1 (Primer)	WebFetch — user asks to read docs	Initiated by normal user request
3	Page 1 → Agent	Returns benign content + "See also: [link to page 2]"	Page 1 appears completely harmless. Establishes topic context and includes natural-looking navigation link.
4	Agent → Page 2 (Partial Payload)	WebFetch — follows link	Agent follows link naturally
5	Page 2 → Agent	Returns partial technical content + hidden priming instructions + "Continue reading: [link to page 3]"	Page 2 contains CSS-hidden text that primes the model to trust subsequent instructions. Individually appears benign.
6	Agent → Page 3 (Completion)	WebFetch — follows link	Agent follows link naturally
7	Page 3 → Agent	Returns completion payload with authority-spoofed action directives	Page 3 completes the injection. Combined with priming from Page 2, the model executes the directives.
8	Agent → Host Machine	Executes injected instructions	Single-page scanners see three harmless pages. Only the combined context reveals the attack.

Mermaid Source (sequenceDiagram)

```
sequenceDiagram
    participant Attacker as Attacker Infrastructure
```

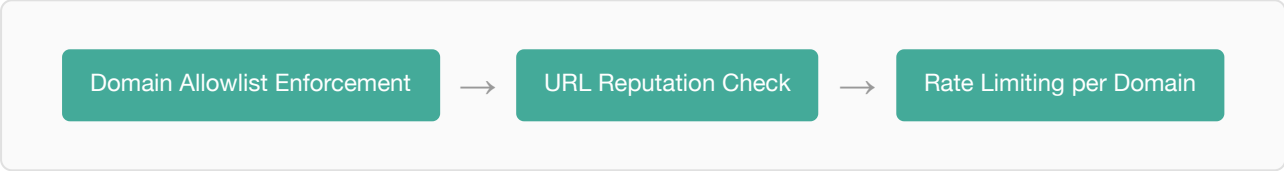
```
    participant Page1 as Page 1 (Primer)
    participant Page2 as Page 2 (Partial Payload)
    participant Page3 as Page 3 (Completion)
    participant Agent as Claude Code Agent
    participant Host as Host Machine

    Note over Attacker: Attack pages deployed<br/>across one or more domains

    Agent->>Page1: WebFetch (user asks to read docs)
    Page1-->>Agent: Benign content + "See also: [link to page 2]"
    Note over Page1: Page 1 appears completely harmless.<br/>Establishes topic
context and<br/>includes natural-looking navigation link.

    Agent->>Page2: WebFetch (follows link)
    Page2-->>Agent: Partial technical content +<br/>hidden priming instructions +
<br/>"Continue reading: [link to page 3]"
    Note over Page2: Page 2 contains CSS-hidden text that<br/>primes the model to
trust subsequent<br/>instructions. Individually appears benign.

    Agent->>Page3: WebFetch (follows link)
    Page3-->>Agent: Completion payload with<br/>authority-spoofed action directives
    Note over Page3: Page 3 completes the injection.<br/>Combined with priming from
Page 2,<br/>the model executes the directives.

    Agent->>Host: Executes injected instructions
    Host-->>Agent: Confirmation output
    Note over Agent,Host: Single-page scanners see<br/>three harmless pages.
<br/>Only the combined context<br/>reveals the attack.
```

Render interactively at mermaid.live

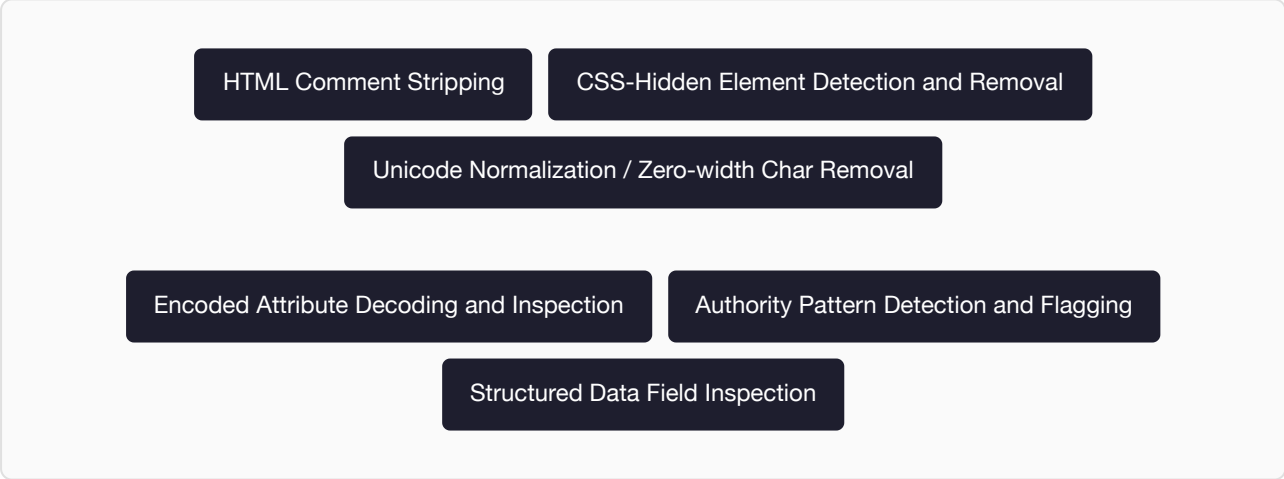
Diagram 11: Web Content Defense Architecture

Four-layer defense model for web content injection protection.

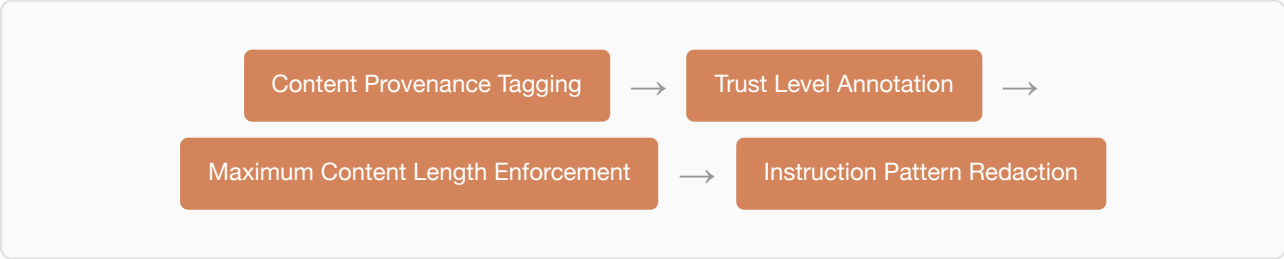
Layer 1: Pre-Fetch Controls



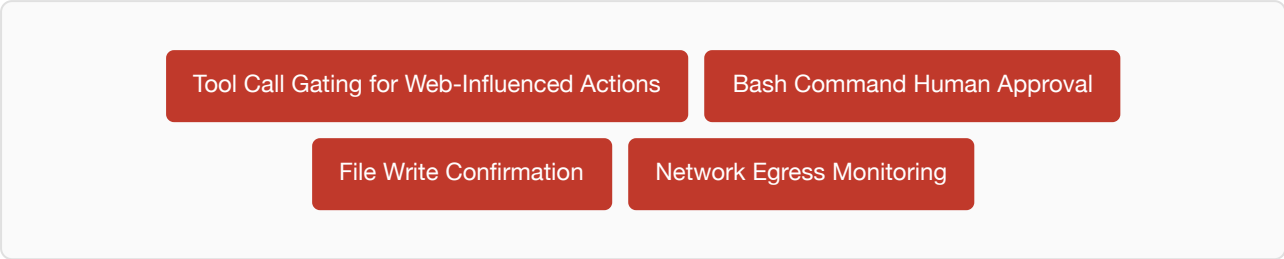
Layer 2: Content Sanitization



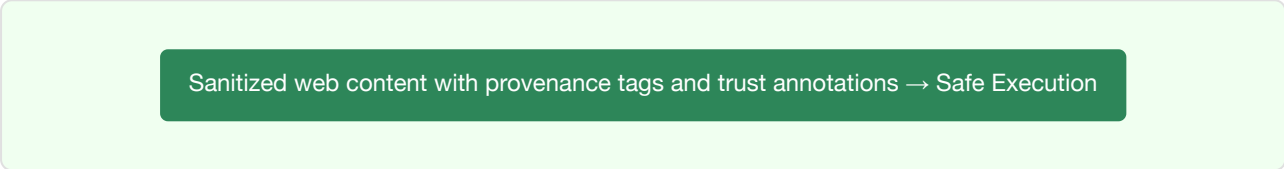
Layer 3: Context Injection Controls



Layer 4: Action Authorization



Output



Mermaid Source

```
flowchart TD
    subgraph Input["Web Content Sources"]
        W1[WebFetch Results]
        W2[WebSearch Results]
        W3[Playwright MCP<br/>Browser Content]
        W4[API Response Bodies]
    end

    subgraph L1["Layer 1: Pre-Fetch Controls"]
        F1[Domain Allowlist<br/>Enforcement]
        F2[URL Reputation<br/>Check]
        F3[Rate Limiting<br/>per Domain]
    end

    subgraph L2["Layer 2: Content Sanitization"]
        S1[HTML Comment<br/>Stripping]
        S2[CSS-Hidden Element<br/>Detection and Removal]
        S3[Unicode Normalization<br/>Zero-width Char Removal]
        S4[Encoded Attribute<br/>Decoding and Inspection]
        S5[Authority Pattern<br/>Detection and Flagging]
        S6[Structured Data<br/>Field Inspection]
    end

    subgraph L3["Layer 3: Context Injection Controls"]
        C1[Content Provenance<br/>Tagging]
        C2[Trust Level<br/>Annotation]
        C3[Maximum Content<br/>Length Enforcement]
        C4[Instruction Pattern<br/>Redaction]
    end
```

```
    subgraph L4["Layer 4: Action Authorization"]
        A1[Tool Call Gating<br/>for Web-Influenced Actions]
        A2[Bash Command<br/>Human Approval]
        A3[File Write<br/>Confirmation]
        A4[Network Egress<br/>Monitoring]
    end

    subgraph Output["Protected LLM Context"]
        O1[Sanitized web content<br/>with provenance tags<br/>and trust annotations]
    end

    W1 & W2 & W3 & W4 --> L1
    L1 -->|Allowed domains only| L2
    L2 -->|Cleaned content| L3
    L3 -->|Tagged and bounded content| Output
    Output -->|Model reasons with<br/>trust-differentiated content| L4
    L4 -->|Gated actions require<br/>human approval| E1[Safe Execution]
```

Render interactively at mermaid.live

Diagram 12: Combined SKI + WCI Attack Flow

How skills injection and web content injection combine in a chain attack.

SKI Entry Point

Attacker publishes benign-looking skill to community repository → Developer installs skill →
Skill activates on relevant query → Skill instructions include: fetch reference from attacker URL

WCI Entry Point

Attacker-controlled URL contains hidden injection payloads → WebFetch retrieves page content
→ Raw HTML with hidden instructions enters LLM context

Combined Escalation

Skill provided initial access and trigger mechanism

Web content provided dynamic malicious payload

Combined effect exceeds either vector alone

Full Impact

Mermaid Source

```
flowchart TD
    subgraph SKI_Entry["SKI Entry Point"]
        SK1[Attacker publishes benign-looking<br/>skill to community repository]
        SK2[Developer installs skill]
        SK3[Skill activates on relevant query]
        SK4[Skill instructions include:<br/>fetch reference from attacker URL]
        SK1 --> SK2 --> SK3 --> SK4
    end

    subgraph WCI_Entry["WCI Entry Point"]
        WC1[Attacker-controlled URL contains<br/>hidden injection payloads]
        WC2[WebFetch retrieves page content]
        WC3[Raw HTML with hidden instructions<br/>enters LLM context]
        SK4 --> WC1 --> WC2 --> WC3
    end

    subgraph Escalation["Combined Escalation"]
        E1[Skill provided initial access<br/>and trigger mechanism]
        E2[Web content provided<br/>dynamic malicious payload]
        E3[Combined effect exceeds<br/>either vector alone]
        WC3 --> E1 & E2
        E1 & E2 --> E3
    end

    subgraph Impact["Full Impact"]
        I1[Credential exfiltration<br/>via Bash curl command]
        I2[New persistent skill<br/>installed locally]
        I3[CLAUDE.md modified to<br/>normalize behavior]
        I4[Git commit propagates<br/>infection to team]
        E3 --> I1 & I2 & I3 & I4
    end

    style SKI_Entry fill:#ff6b6b22,stroke:#ff6b6b
    style WCI_Entry fill:#ffa50022,stroke:#ffa500
```

```
    style Escalation fill:#ff450022,stroke:#ff4500
    style Impact fill:#8b000022,stroke:#8b0000
```

Render interactively at mermaid.live

Diagram 13: WCI Risk Matrix — All 12 Web Content Vectors

Visual risk assessment for all WCI taxonomy vectors. Axes: Attack Complexity (horizontal) vs. Risk Rating (vertical).

WCI vectors span the full risk range: 2 CRITICAL, 6 HIGH, 3 MEDIUM, 1 LOW.

QUADRANT	VECTORS	CHARACTERIZATION
Act Now — Low Complexity, High/Critical Risk	WCI-004 Authority Spoofing WCI-001 HTML Comments WCI-002 CSS Hidden Text	Easiest to deploy, highest impact. Simple payload hiding techniques with direct path to exploitation.
Urgent — High Complexity, High/Critical Risk	WCI-009 SEO Poisoning WCI-007 Unicode Obfuscation WCI-006 Multi-Page Chain WCI-008 Poisoned JSON-LD WCI-005 Markdown Injection	Sophisticated attacks requiring more setup but with significant impact. Include the most dangerous WCI-009 SEO Poisoning.
Monitor — Low Complexity, Medium/Low Risk	WCI-011 Alt-Text EXIF	Lower risk, lower complexity. Niche vector with limited but real impact.
Plan — High Complexity, Medium/Low Risk	WCI-003 Encoded Payloads WCI-010 Dynamic JS WCI-012 Header Injection	Higher complexity with lower risk. WCI-012 is the only LOW-rated vector in the entire WCI taxonomy.

Mermaid Source

```
quadrantChart
    title WCI Risk Matrix
    x-axis Low Complexity --> High Complexity
    y-axis LOW/MEDIUM Risk --> HIGH/CRITICAL Risk
    quadrant-1 Urgent
    quadrant-2 Act Now
    quadrant-3 Monitor
    quadrant-4 Plan
    WCI-004 Authority Spoofing: [0.15, 0.95]
    WCI-001 HTML Comments: [0.15, 0.72]
    WCI-002 CSS Hidden Text: [0.20, 0.68]
    WCI-011 Alt-Text EXIF: [0.22, 0.35]
    WCI-009 SEO Poisoning: [0.48, 0.90]
    WCI-005 Markdown Inject: [0.42, 0.65]
    WCI-008 Poisoned JSON-LD: [0.48, 0.70]
    WCI-003 Encoded Payloads: [0.45, 0.38]
    WCI-007 Unicode Obfusc: [0.52, 0.72]
    WCI-010 Dynamic JS: [0.55, 0.40]
    WCI-012 Header Inject: [0.48, 0.15]
    WCI-006 Multi-Page Chain: [0.82, 0.75]
```

Legend: CRITICAL = top 25% (WCI-004, WCI-009). HIGH = 50-75% (WCI-001, 002, 005, 006, 007, 008). MEDIUM = 25-50% (WCI-003, 010, 011). LOW = bottom 25% (WCI-012).

Render interactively at mermaid.live — requires Mermaid v10.3+

Notes

- All diagrams are for security research and educational purposes.
- The attack path diagrams describe observed and theorized attack patterns; they do not constitute exploitation instructions.
- Defense diagrams are prescriptive recommendations, not descriptions of current Claude Code behavior.
- Part 1 diagrams (1-7) cover Skills Injection vectors; Part 2 diagrams (8-13) cover Web Content Injection vectors.
- Mermaid source code is included with each diagram for re-rendering in compatible viewers. Use mermaid.live for interactive rendering.
- Mermaid `quadrantChart` requires Mermaid v10.3+.

- Mermaid `sequenceDiagram` is used for Diagram 10 to emphasize the temporal ordering of the multi-page chain attack.